Day 8 – GitHub Repository Finalization and Phase 1 Closure

1. Project Status Overview

Day 8 focused on finalizing the GitHub repository and officially closing **VisionGuard Phase 1**.

After Day 7 prepared the folder structure, documentation, architecture diagram, and GitHub-ready files, Day 8 involved uploading the project, improving the repository presentation, adding visual elements to the README, confirming clone/download readiness, and preparing the roadmap for Phase 2 using ESP32.

This day marked the transition from:

```
Local Project Folder
```

to:

```
Public GitHub Portfolio Project
```

---

2. GitHub Repository Created

A new GitHub repository was created for the VisionGuard project.

Repository Name

```
VisionGuard
```

Repository Description

```
AI-powered intruder detection system using Python, OpenCV, and Arduino Uno.
It detects motion and faces in real time, triggers LED/buzzer hardware
alerts via serial communication, saves evidence images, and includes motion
visualization and cooldown-based alert control.
```

The description clearly summarizes the project's purpose, technologies, and core functionality within a professional GitHub format.

---

3. Project Uploaded to GitHub

The complete VisionGuard Phase 1 project folder was uploaded to GitHub.

Uploaded Content

```
Python source code
Arduino sketches
Technical reports
Architecture diagram
Prototype circuit photos
Complete demo photos
README.md
requirements.txt
.gitignore
Sample capture image
Phase 2 roadmap
```

Repository Verification Checklist

The GitHub repository was checked to confirm:

```
README visible on main page
code/ folder uploaded
arduino/ folder uploaded
docs/ folder uploaded
images/ folder uploaded
captures/ folder uploaded with sample image
requirements.txt present
.gitignore present
architecture diagram visible
prototype images opening correctly
```

Result

```
GitHub repository verified successfully.
```

---

4. README Improved with Images

The README file was upgraded by adding visual sections for better presentation.

Added README Sections

```
## Phase 1 Architecture

![Phase 1 Architecture](images/phase_1_architecture.png)

## Hardware Prototype

![Prototype Circuit](images/prototype_circuit.jpg)
```

```
## Working Demo

![Complete Demo](images/complete_demo.jpg)
```

Purpose of Adding Images

The images make the repository more visually understandable and professional.

They help viewers immediately understand:

```
System architecture
Arduino hardware setup
Real working prototype
AI detection output
```

This improves the first impression of the GitHub project.

---

5. Phase 1 Completion Status Added

A dedicated Phase 1 completion section was added to the README.

Added Section

```
## Phase 1 Status

Phase 1 of VisionGuard has been completed successfully. The system detects
motion and faces using Python and OpenCV, sends serial commands to Arduino
Uno, and triggers LED and buzzer hardware alerts in real time.

Current Phase 1 capabilities:
- Real-time motion detection
- Face detection using OpenCV
- Motion memory for stable alerts
- Arduino Uno LED and buzzer control
- Evidence image capture with cooldown
- Motion visualization on video feed
```

Importance

This section clearly communicates that Phase 1 is complete and functional. It also separates the current working system from future planned improvements.

---

6. Clone / Download Readiness Checked

The repository was reviewed from the perspective of a new user who wants to download and run the project.

Checked Items

```
requirements.txt exists
Arduino code exists
Main Python code exists
COM port instruction is mentioned
Arduino setup instructions are clear
Run command is mentioned
Exit key instruction is present
```

README Sections Checked

```
## Installation

pip install -r requirements.txt
## Arduino Setup

1. Upload arduino/command_mode.ino to Arduino Uno.
2. Connect LED to pin 13.
3. Connect active buzzer to pin 8.
4. Select correct COM port in Arduino IDE.
## Run the Project

1. Connect Arduino Uno to laptop.
2. Close Arduino Serial Monitor.
3. Open code/visionguard_main.py.
4. Change COM port if needed.
5. Run the Python script.
6. Press q to exit.
```

Result

```
Repository clone/download readiness checked successfully.
```

This ensures that another person can understand and run the project without needing direct explanation.

---

7. Phase 2 ESP32 Roadmap Added

A Phase 2 roadmap was prepared and added to the repository.

File Created

```
docs/Phase_2_Roadmap.md
```

Phase 2 Title

```
# VisionGuard Phase 2 Roadmap – ESP32 IoT Alert System
```

Phase 2 Goal

```
Upgrade VisionGuard from local Arduino-based alerting to an ESP32-based IoT
alert system with WiFi-enabled remote notification capability.
```

Planned Phase 2 Tasks

```
ESP32 setup
ESP32 LED + buzzer test
WiFi connection test
Python to ESP32 communication
Remote alert system
Future expansion
```

Roadmap Content

```
# VisionGuard Phase 2 Roadmap – ESP32 IoT Alert System

## Goal

Upgrade VisionGuard from local Arduino-based alerting to an ESP32-based IoT
alert system with WiFi-enabled remote notification capability.

## Phase 2 Tasks

### 1. ESP32 Setup
- Install ESP32 board support in Arduino IDE
- Select correct ESP32 board and COM port
- Upload blink test code

### 2. ESP32 LED + Buzzer Test
- Connect LED and active buzzer to ESP32
- Test GPIO-based alert control

### 3. WiFi Connection Test
- Connect ESP32 to local WiFi
- Print connection status and IP address on Serial Monitor

### 4. Python to ESP32 Communication
- Send alert signal from Python to ESP32
- Test serial or WiFi-based command control

### 5. Remote Alert System
- Add Telegram or email notification
- Send alert message when intruder is detected

### 6. Future Expansion
- Web dashboard
- Cloud image storage
- Authorized vs unauthorized face recognition
```

---

8. GitHub Website Workflow Used

The repository updates were made directly using the GitHub website.

Workflow Used

```
Open repository
Select file to edit
Click pencil/edit icon
Make changes
Scroll to Commit changes
Write commit message
Commit directly to main branch
```

For creating a new roadmap file:

```
Add file
→ Create new file
→ docs/Phase_2_Roadmap.md
→ Paste roadmap content
→ Commit changes
```

This was useful because changes could be made directly without needing local Git commands.

9. Phase 1 Final Status

VisionGuard Phase 1 is now complete.

Completed Phase 1 System

```
Laptop camera
→ Python OpenCV motion detection
→ Face detection
→ Motion memory
→ Decision logic
→ PySerial communication
→ Arduino Uno R3
→ LED + buzzer hardware alert
→ Evidence image capture
→ Motion visualization
```

Current Capabilities

```
Real-time webcam monitoring
Motion detection
Face detection
Motion memory for stable triggering
Arduino Uno serial communication
LED and buzzer alert
Evidence capture with cooldown
Motion area visualization
Professional documentation
GitHub-ready repository
```